Universidad Latina de Costa Rica

Sistemas Operativos II

Título

Proxy HTTP con Caché: Implementación y Rendimiento

Tipo de informe:

Informe para Proyecto Final

Profesor:

Carlos Méndez Rodríguez

Estudiantes:

José Araya Fernández

Amy Morales Cruz

Primer Período Lectivo 2026

Índice

Resumen 2
Introducción 3
Objetivos 4
Objetivo General 5
Objetivos Específicos 5
Marco Teórico 5
Protocolo HTTP 6
Servidores Proxy 6
Caché en Sistemas de Red 6
Sockets y Concurrencia en Go 7
Metodología 7
Resultados 9
Discusión 10
Conclusión 11
Anexos 12
Bibliografía 13

Resumen

Introducción

Los servidores proxy HTTP actúan como intermediarios entre los clientes y los servidores de origen, recibiendo las solicitudes de los usuarios y gestionando su respuesta de manera eficiente. Uno de los mecanismos más importantes que implementan es el almacenamiento en caché, mediante el cual guardan copias locales de los recursos solicitados para poder servirlos directamente en futuras peticiones, reduciendo así el tráfico hacia los servidores de origen y mejorando los tiempos de respuesta (IBM, 2015). Este enfoque resulta especialmente relevante en entornos de alta demanda, donde la optimización del rendimiento y la reducción de la carga en los servidores son aspectos críticos.

El proceso de caché en un proxy HTTP sigue una lógica bien definida: cuando un cliente solicita un recurso, el proxy primero verifica si existe una copia válida en su caché; si la encuentra y está vigente, la entrega directamente al cliente (cache hit); si está desactualizada, la reválida con el servidor de origen; y si no existe, la descarga, la entrega y la almacena para solicitudes futuras (cache miss) (Apache Software Foundation, 2023). La vigencia de los recursos almacenados se determina mediante encabezados HTTP como Cache-Control, Expires y max-age, los cuales permiten tanto al servidor de origen como al administrador del proxy definir políticas precisas de almacenamiento.

El presente proyecto tiene como objetivo diseñar e implementar un proxy HTTP con caché en Java, aplicando conceptos de sockets TCP, concurrencia y manejo de sistemas de archivos. A través de esta implementación se busca analizar el impacto del caché en el rendimiento del sistema, evaluando métricas como el tiempo de respuesta, la tasa de aciertos de caché y el comportamiento bajo carga de múltiples clientes simultáneos.

Objetivos

Objetivo General

Implementar un servidor proxy HTTP con caché en Java que permita reenviar solicitudes a servidores de origen, optimizando el rendimiento mediante el almacenamiento local de recursos frecuentemente solicitados.

Objetivos Específicos

1. Implementar un servidor proxy capaz de recibir y procesar solicitudes HTTP utilizando sockets TCP.
2. Desarrollar un mecanismo de caché que almacene respuestas HTTP para reducir el tiempo de respuesta en solicitudes repetidas.
3. Gestionar la concurrencia mediante hilos para atender múltiples clientes de forma simultánea.

Marco Teórico

Protocolo HTTP

El Protocolo de Transferencia de Hipertexto (HTTP, por sus siglas en inglés) es el fundamento de la comunicación de datos en la World Wide Web. Se trata de un protocolo de capa de aplicación basado en un modelo cliente-servidor, en el cual un cliente envía una solicitud y el servidor devuelve una respuesta (Fielding & Reschke, 2014). HTTP define un conjunto de métodos de solicitud que indican la acción deseada sobre un recurso: GET, para recuperar datos; POST, para enviar información al servidor; PUT, para actualizar un recurso; y DELETE, para eliminarlo, entre otros.

Cada mensaje HTTP está compuesto por una línea de inicio, un conjunto de encabezados (headers) y un cuerpo opcional. Los encabezados transmiten metadatos esenciales como el tipo de contenido, la longitud del mensaje, las directivas de caché y la información de autenticación. Las versiones más utilizadas son HTTP/1.1, que introdujo conexiones persistentes y transferencia por fragmentos, y HTTP/2, que mejoró el rendimiento mediante la multiplexación de solicitudes sobre una sola conexión TCP (Mozilla Developer Network, 2023).

Servidores Proxy

Un servidor proxy es un intermediario que se sitúa entre los clientes y los servidores de destino, recibiendo las solicitudes de los primeros y reenvíandolas en su nombre. Este modelo permite implementar funcionalidades adicionales como control de acceso, registro de actividad, balanceo de carga, compresión de contenido y almacenamiento en caché (Luotonen & Altis, 1994). Existen dos tipos principales: el proxy directo (forward proxy), que actúa en nombre del cliente y es transparente para el servidor; y el proxy inverso (reverse proxy), que actúa en nombre del servidor y es transparente para el cliente.

En el contexto de las redes empresariales y académicas, los proxies directos se utilizan frecuentemente para filtrar contenido, reducir el consumo de ancho de banda y mejorar los tiempos de respuesta mediante la reutilización de recursos previamente descargados. IBM (2015) señala que el proxy guarda copias locales de los recursos solicitados, de modo que las peticiones repetidas pueden ser atendidas directamente desde el almacenamiento local sin necesidad de contactar al servidor de origen.

Caché en Sistemas de Red

La caché web es un mecanismo que permite almacenar temporalmente copias de recursos HTTP con el objetivo de reducir la latencia y el tráfico de red. Cuando un recurso se almacena en caché, las solicitudes posteriores pueden ser atendidas localmente, lo que se denomina cache hit. Si el recurso no está almacenado o ha expirado, se produce un cache miss, y el proxy debe contactar al servidor de origen para obtener una versión actualizada (Apache Software Foundation, 2023).

La validez de los recursos almacenados se controla mediante encabezados HTTP específicos. El encabezado Cache-Control permite definir directivas como max-age, que indica el tiempo máximo en segundos durante el cual un recurso se considera válido; no-cache, que obliga al proxy a revalidar el recurso con el servidor antes de servirlo; y no-store, que prohíbe el almacenamiento en caché. El encabezado Expires cumple una función similar, indicando la fecha y hora exacta en la que el recurso deja de ser válido. Cuando un recurso ha expirado, el proxy puede revalidarlo mediante los encabezados condicionales If-Modified-Since o If-None-Match, evitando la transferencia completa del contenido si no ha cambiado (Apache Software Foundation, 2023).

El almacenamiento en caché puede realizarse en memoria RAM, lo que ofrece tiempos de acceso muy bajos pero capacidad limitada, o en disco, que permite mayor capacidad a costa de una mayor latencia. IBM (2015) indica que los sistemas de proxy mantienen un índice en memoria con información sobre las URLs almacenadas, su ubicación y fecha de expiración, con el fin de acelerar la búsqueda de recursos en caché.

Sockets y Concurrencia en Go

Un socket es un punto de comunicación entre dos procesos a través de una red. En Go, el paquete net de la biblioteca estándar proporciona las abstracciones necesarias para establecer conexiones TCP sin necesidad de gestionar directamente los detalles del protocolo. La función net.Listen permite crear un servidor que escucha conexiones entrantes en un puerto específico, mientras que net.Dial establece una conexión hacia un servidor remoto. Una vez establecida la conexión, ambos extremos intercambian datos mediante la interfaz net.Conn, que expone métodos de lectura y escritura compatibles con el modelo de streams de TCP (Go Authors, s.f.).

Para atender múltiples clientes de forma simultánea, Go utiliza goroutines: funciones ligeras gestionadas por el runtime del lenguaje que permiten ejecutar tareas concurrentes con un costo de memoria significativamente menor al de los hilos del sistema operativo. El patrón habitual consiste en invocar go handleClient(conn) por cada conexión aceptada, delegando su atención a una goroutine independiente sin bloquear el bucle principal del servidor. La comunicación y sincronización entre goroutines se realiza mediante canales (channels), siguiendo la filosofía del lenguaje: "Do not communicate by sharing memory; instead, share memory by communicating" (Go Authors, s.f.). Este modelo resulta especialmente adecuado para un proxy HTTP bajo alta carga, ya que permite gestionar miles de conexiones concurrentes con un uso eficiente de los recursos del sistema.

Metodología

Para el desarrollo del proyecto se adoptó una metodología ágil e iterativa basada en sprints, ampliamente utilizada en proyectos de desarrollo de software por su capacidad de adaptación y entrega continua de valor (Wake Forest University, s.f.). Cada sprint corresponde a un ciclo de trabajo de aproximadamente una semana, al final del cual se obtiene un entregable funcional que sirve de base para la siguiente iteración. Este enfoque permite validar la comunicación entre sockets antes de añadir la complejidad de la caché, reduciendo el riesgo de errores acumulados.

Las fases del proyecto se organizaron de la siguiente manera:

Fase / Sprint	Actividades	Entregable
1. Configuración del servidor	Configuración del ServerSocket y manejo de peticiones mediante hilos (threads).	Servidor capaz de aceptar múltiples conexiones.
2. Implementación del protocolo HTTP	Análisis (parsing) de cabeceras HTTP con soporte para métodos GET y HOST.	Proxy funcional sin caché.
3. Gestión del caché	Implementación de una estructura de datos para almacenamiento local con lógica de expiración.	Sistema de almacenamiento en memoria o disco.
4. Pruebas y ajustes	Pruebas de concurrencia y validación de tiempos de respuesta con herramientas de benchmarking.	Informe de rendimiento y código final.

El manejo de protocolos de red a bajo nivel suele presentar errores inesperados; trabajar por iteraciones permite asegurar que el flujo de datos (Cliente → Proxy → Internet) sea estable antes de intervenir la respuesta para almacenarla. Con un enfoque experimental, se realizarán pruebas comparativas de tiempo de carga con y sin caché para demostrar empíricamente la optimización del rendimiento.

Las herramientas utilizadas para el desarrollo son las siguientes: el lenguaje de programación Go en su versión estable más reciente, empleando el paquete net de la biblioteca estándar para la comunicación TCP y el paquete sync junto con goroutines para la gestión de concurrencia. Como entorno de desarrollo se utilizará WSL2 con Ubuntu 24.04 bajo Windows, que provee un entorno Linux nativo adecuado para el desarrollo y prueba de aplicaciones de red. El proyecto se alojará en un repositorio público de GitHub bajo licencia MIT como parte de los requisitos de entrega, e incluirá un archivo README con instrucciones de configuración y despliegue. Para la distribución y ejecución reproducible del proxy se utilizará Docker, de modo que el sistema pueda desplegarse en cualquier entorno sin dependencias adicionales. Para las pruebas se emplearán cURL y Postman para validar solicitudes HTTP individuales, un

navegador web para pruebas de navegación real, y Apache JMeter para simular múltiples clientes concurrentes y medir el comportamiento del servidor bajo carga (LoadView, s.f.).

Resultados

Discusión

Conclusión

Anexos

Bibliografía

- Apache Software Foundation. (s.f.). *HTTP Proxy Caching*. Apache Traffic Server. https://docs.trafficserver.apache.org/admin-guide/configuration/cache-basics.en.html
- IBM. (2026). *Overview of proxy server caching*. IBM WebSphere Application Server Network Deployment 8.5.5. https://www.ibm.com/docs/en/was-nd/8.5.5?topic=caching-overview-proxy-server
- LoadView. (s. f.). *Pruebas de carga: Las 10 mejores herramientas de código abierto y de pago*. https://www.loadview-testing.com/es/blog/pruebas-de-carga-revisadas-las-10-mejores-herramientas-de-prueba-de-carga-de-codigo-abierto-y-de-pago/
- Wake Forest University. (s. f.). *Benefits of agile project management*. School of Professional Studies. https://sps.wfu.edu/articles/benefits-agile-project-management/
- Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing (RFC 7230).* Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7230
- Luotonen, A., & Altis, K. (2003). *World-Wide Web proxies.* Computer Networks and ISDN Systems, 27(2), 147-154. https://doi.org/10.1016/0169-7552(94)90128-7
- Mozilla Developer Network. (2023). *HTTP*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP
- Go Authors. (s.f.). *The Go programming language specification — Package net*. Google. https://pkg.go.dev/net